

O'REILLY®

Advanced Harness Engineering for AI Agents





ADVANCED COURSE

Advanced Harness Engineering for AI Agents

Application modes · semantic and ontology layers · durable workflows · deep research · meta-harnesses · delivery surfaces

Meet your Instructor

Richmond Alake

- **Director of AI Developer Experience at Oracle:** Leads AI developer experience and outreach for Oracle AI Database, focusing on open-source framework integrations, AI ecosystem, and developer enablement.
- **AI Engineer & Computer Vision Specialist:** Holds an MSc in Computer Vision, Machine Learning and Robotics from the University of Surrey, with software development and ML engineering experience since around 2016. Former ML Architect at Slalom Build and AI/ML Developer Advocate at MongoDB.
- **Technical Writer & Educator:** Authored 200+ technical articles with over 1 million views across Medium, MongoDB Developer Hub, and the NVIDIA Developer Blog. Co-instructor of a DeepLearning.AI short course alongside Andrew Ng's team and lead instructor for an O'Reilly live training bootcamp.
- **Memory Engineering Pioneer:** Coined "Memory Engineering" as a named AI discipline and built a sustained public identity around it through the #100DaysOfAgentMemory series.
- **AI Stack Engineer:** Creator of MemoRizz (open-source agent memory library) and the Oracle AI Agent Memory Package.





Course Schedule



Module 1: Harness Engineering Continued

Module 2: What is Advanced Harness Engineering

Module 3: Application Mode and Mindset

Module 4: Evaluation and Security

Module 1: Harness Engineering Continued





Harness Engineering Overview

AI agent harness, defined

THE WORKING DEFINITION

agent harness */ˈeɪ.dʒənt ˈhɑː.nəs/ noun*

also **agentic harness**; the software layer around a model. cf. **test harness** in software engineering

- 1 The **software layer surrounding a language model** that turns raw capability into reliable, autonomous action — marshalling its inputs, managing its context and memory, exposing the tools it can act through, and governing the loop in which it **perceives, reasons, acts, and persists** until a goal is met.
- 2 *(engineering sense)* A **modular, configurable system** whose components can be tuned and searched — making the harness itself the unit of optimisation, independent of the model.

agent = model + harness

WHAT THE HARNESS SUPPLIES

INPUTS

perception & grounding

CONTEXT

memory & the window

ACTION

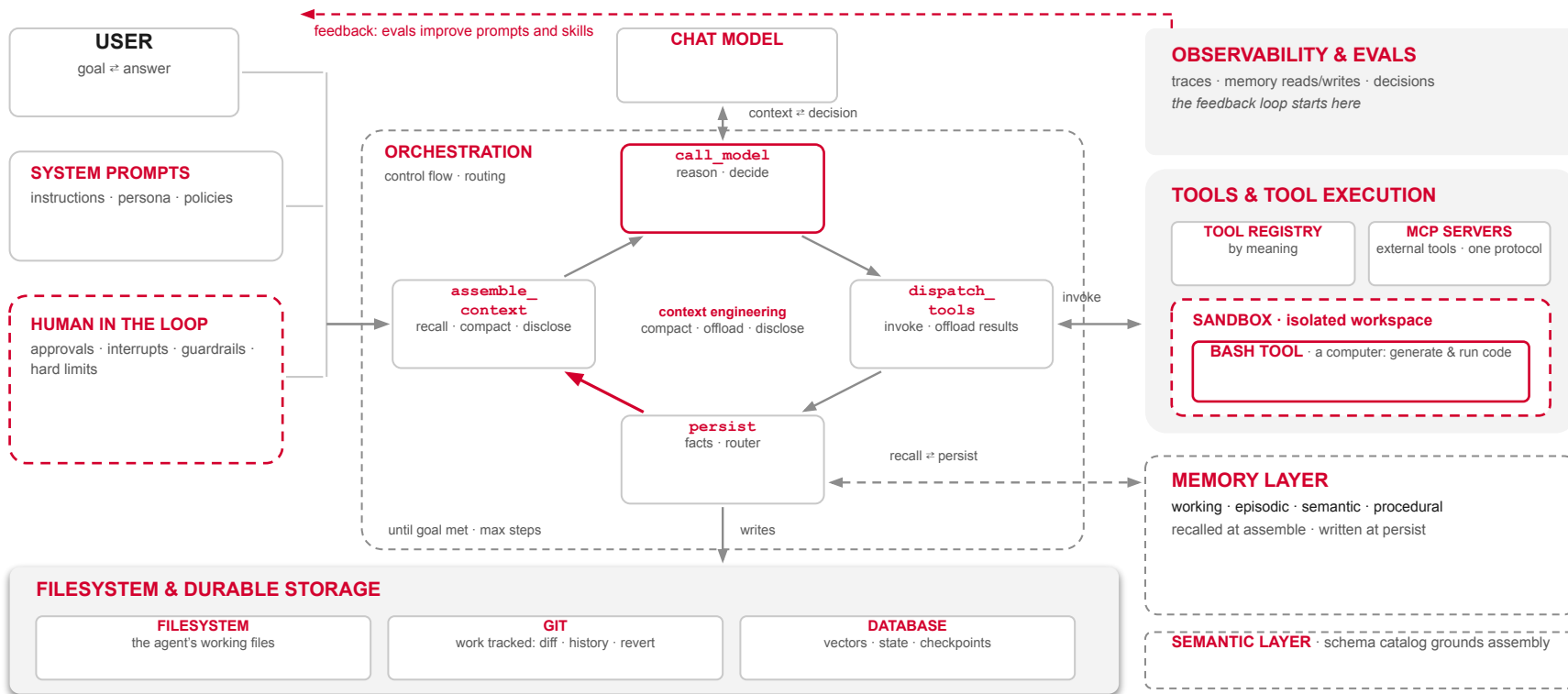
tools · skills · sandbox

CONTROL

the loop & its guardrails

none of it lives in the model

The agent harness, a reference architecture



Module 2: What is Advanced Harness Engineering





The Advanced Learning Path

Content Map · Opening

01 Application mode

assistant · workflow · deep research · hybrid transitions

02 Harness anatomy

nine planes from ingress to evaluation

03 Durable systems

checkpointing · idempotency · approvals · evidence

04 Meta-harnesses

common contract · routing · comparison · optimisation

05 Ecosystem

DeepSeek · OpenClaw · Pi · Omnigent · native SDKs

06 Delivery shapes

CLI · SDK · headless · protocols · UI / IDE / chat

07 Harness evaluation

Terminal-Bench · Harbor · component tests · fair attribution

08 Production proof

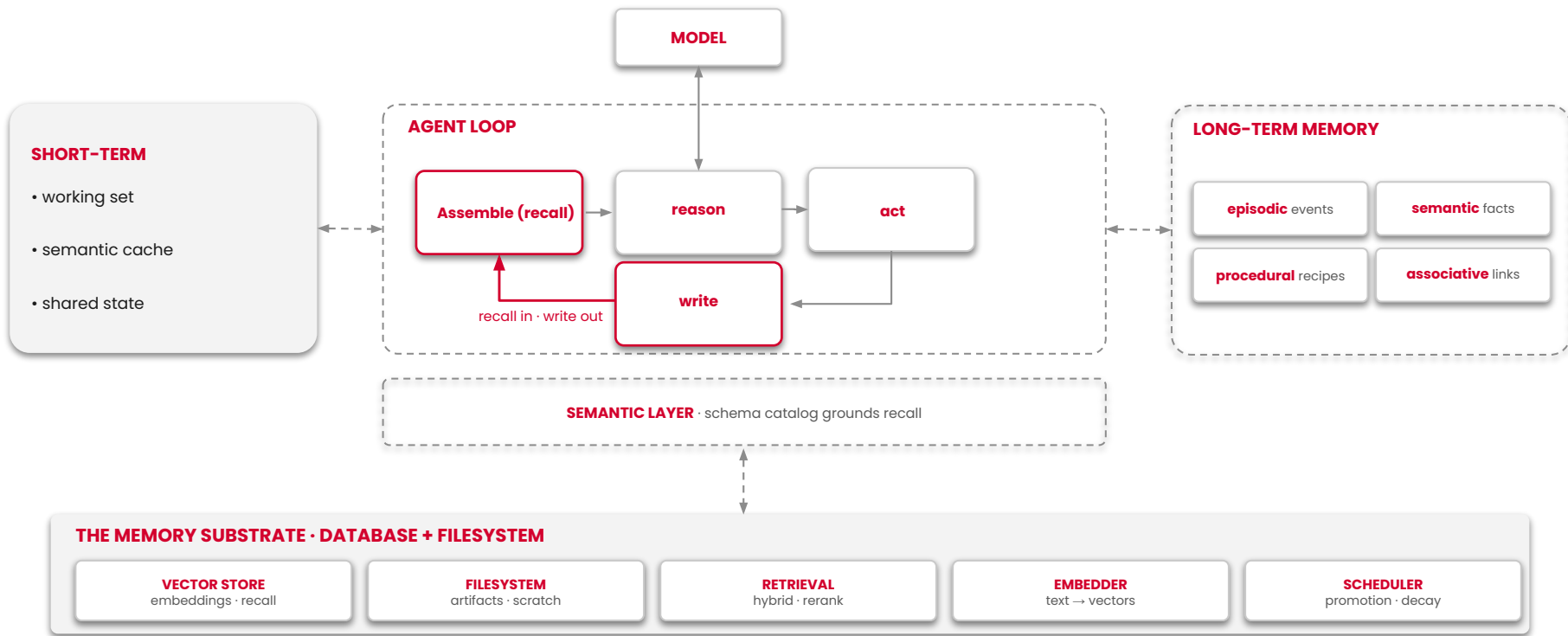
threat model · E2B · evals · rollout · capstone

IN THE LOOP From application intent to architecture, governance, delivery, evaluation and proof. The ecosystem section is a dated snapshot, not a leaderboard.



Having An Implementation Opinion

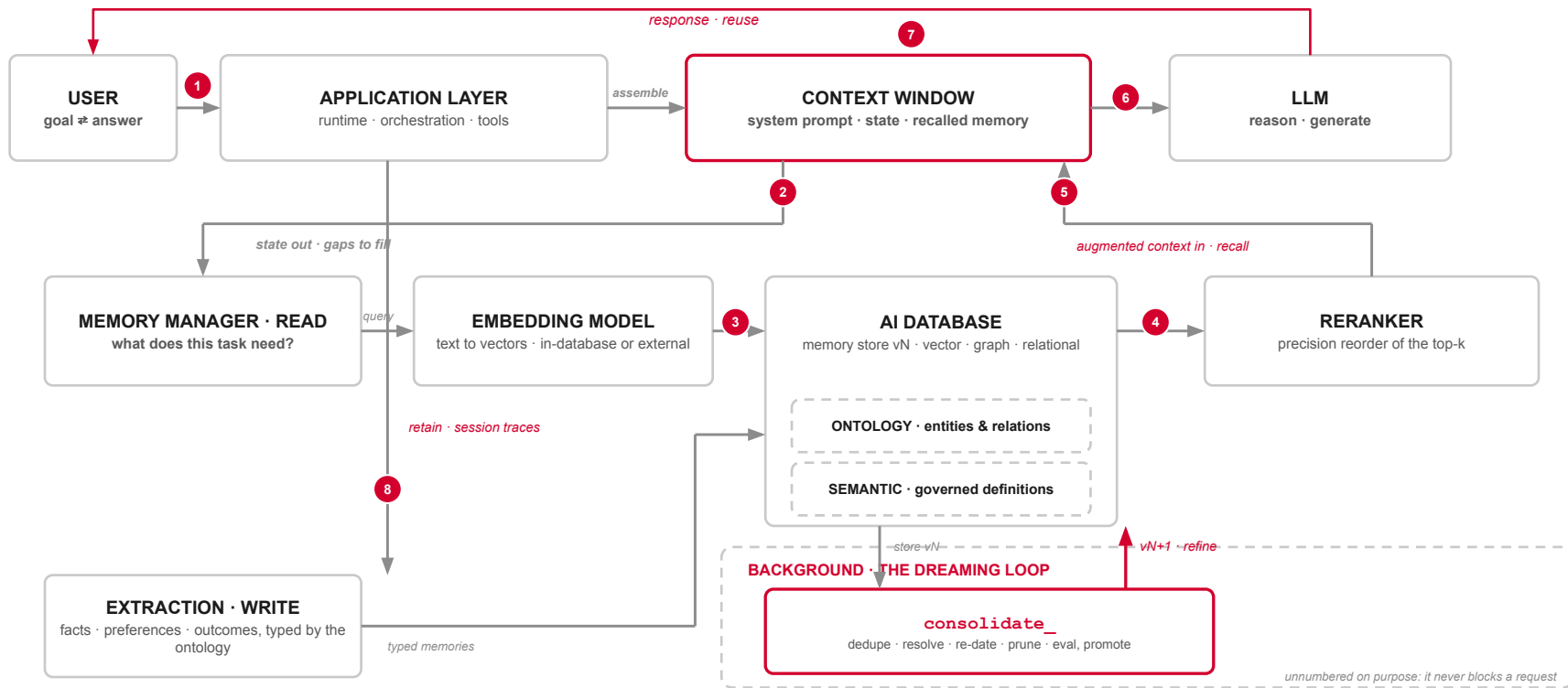
The memory-first harness, in full





Agent Memory, A Reference Architecture

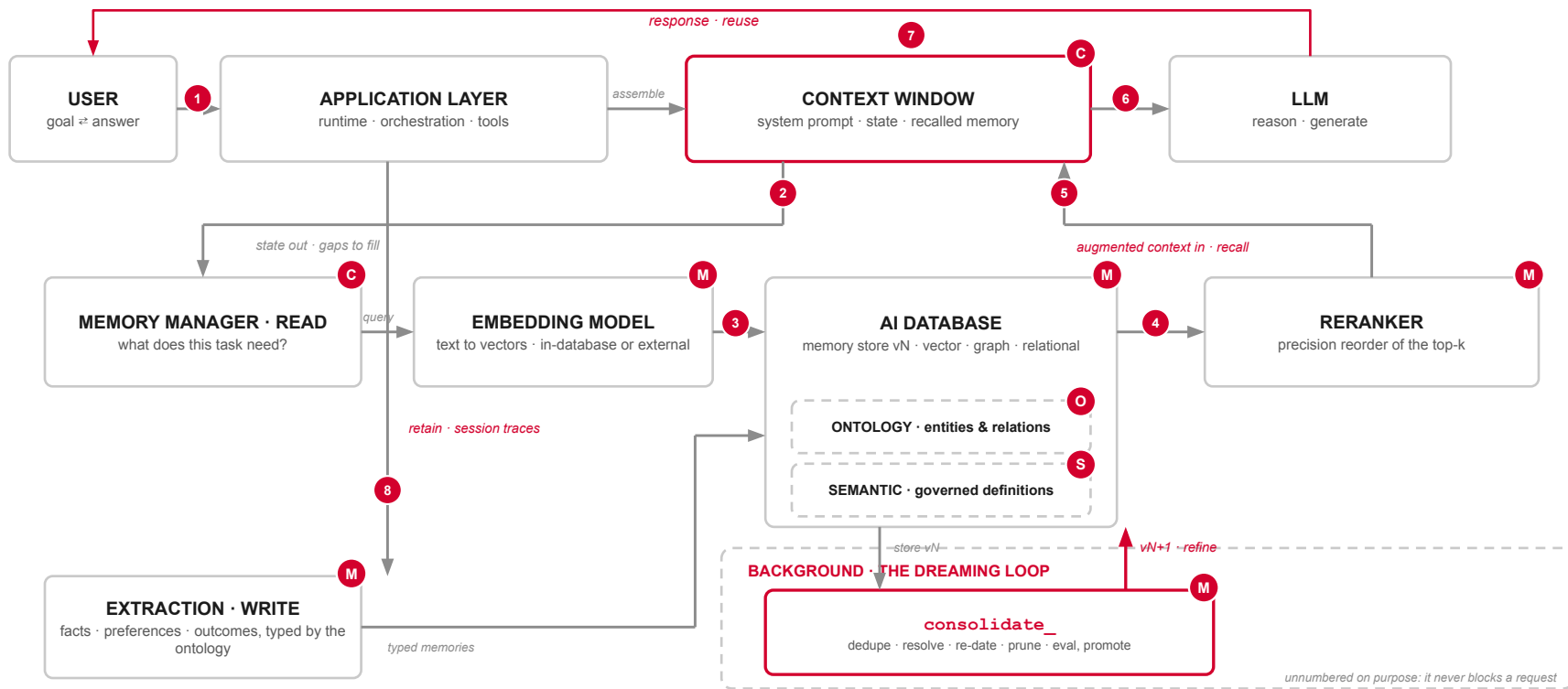
Overview of AI Memory





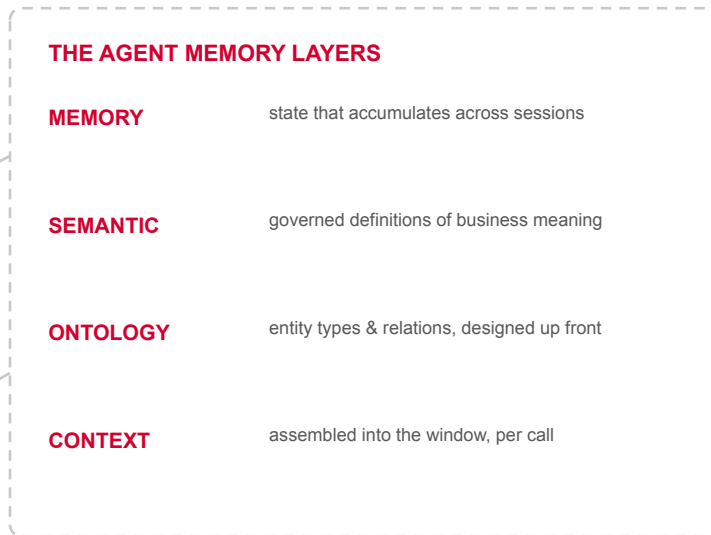
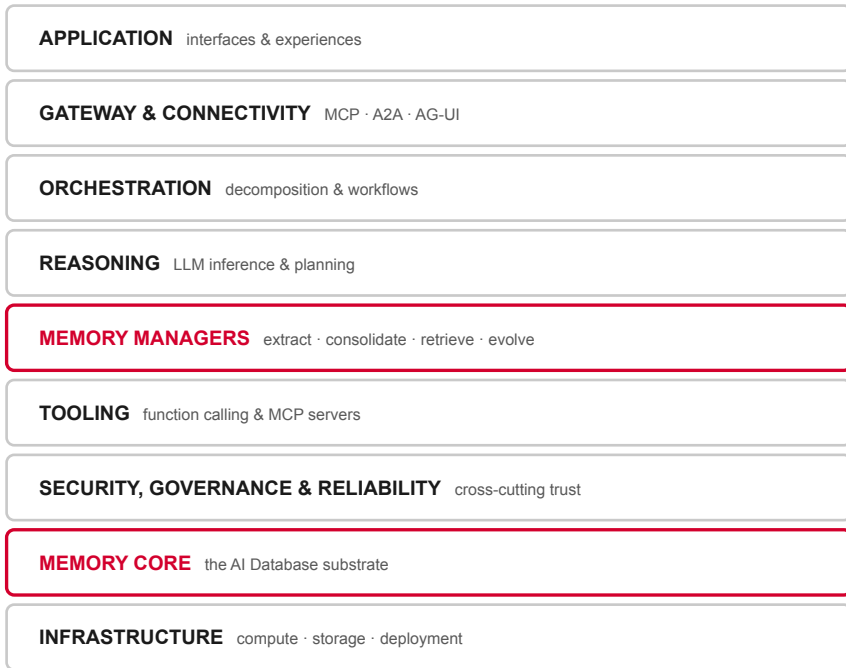
Where The Layers Live

Mapping the Agent Stack onto the architecture



The Agent Stack

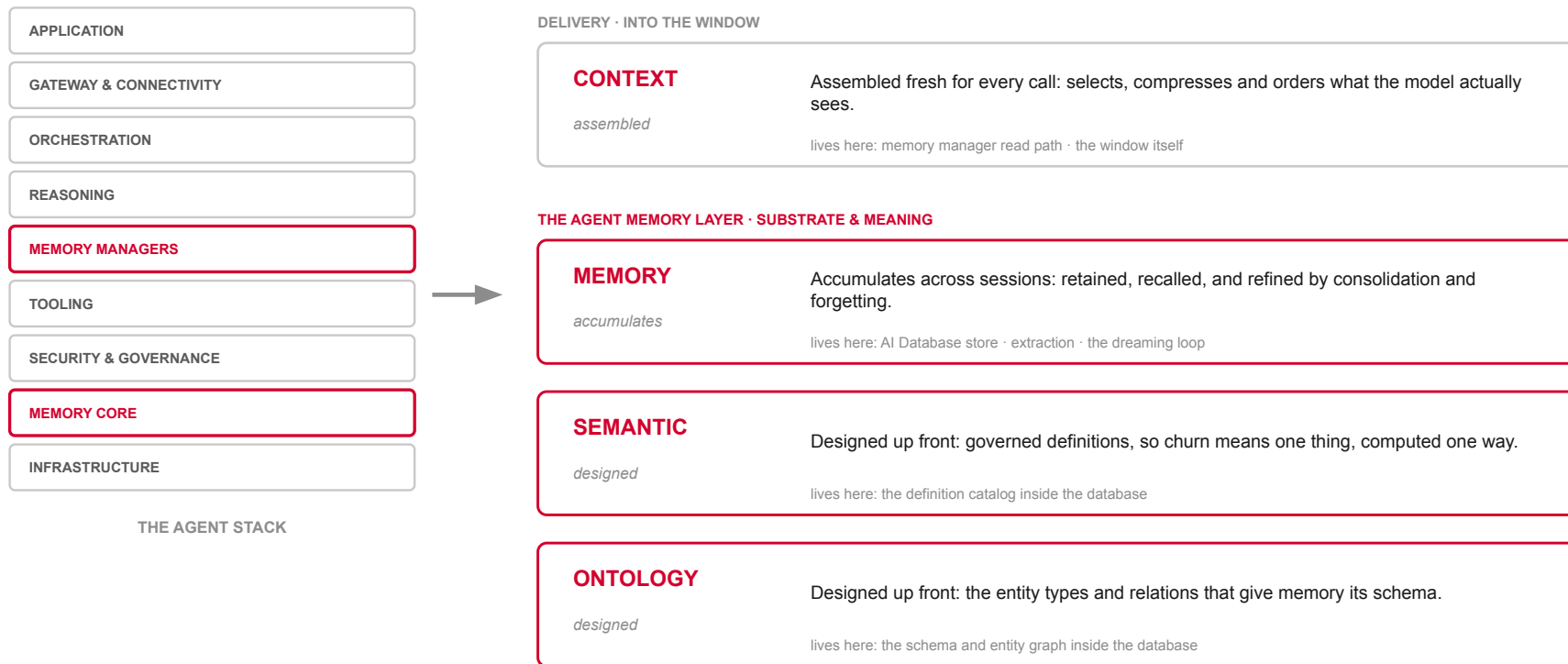
Overview of AI Memory





Inside The Agent Memory Layer

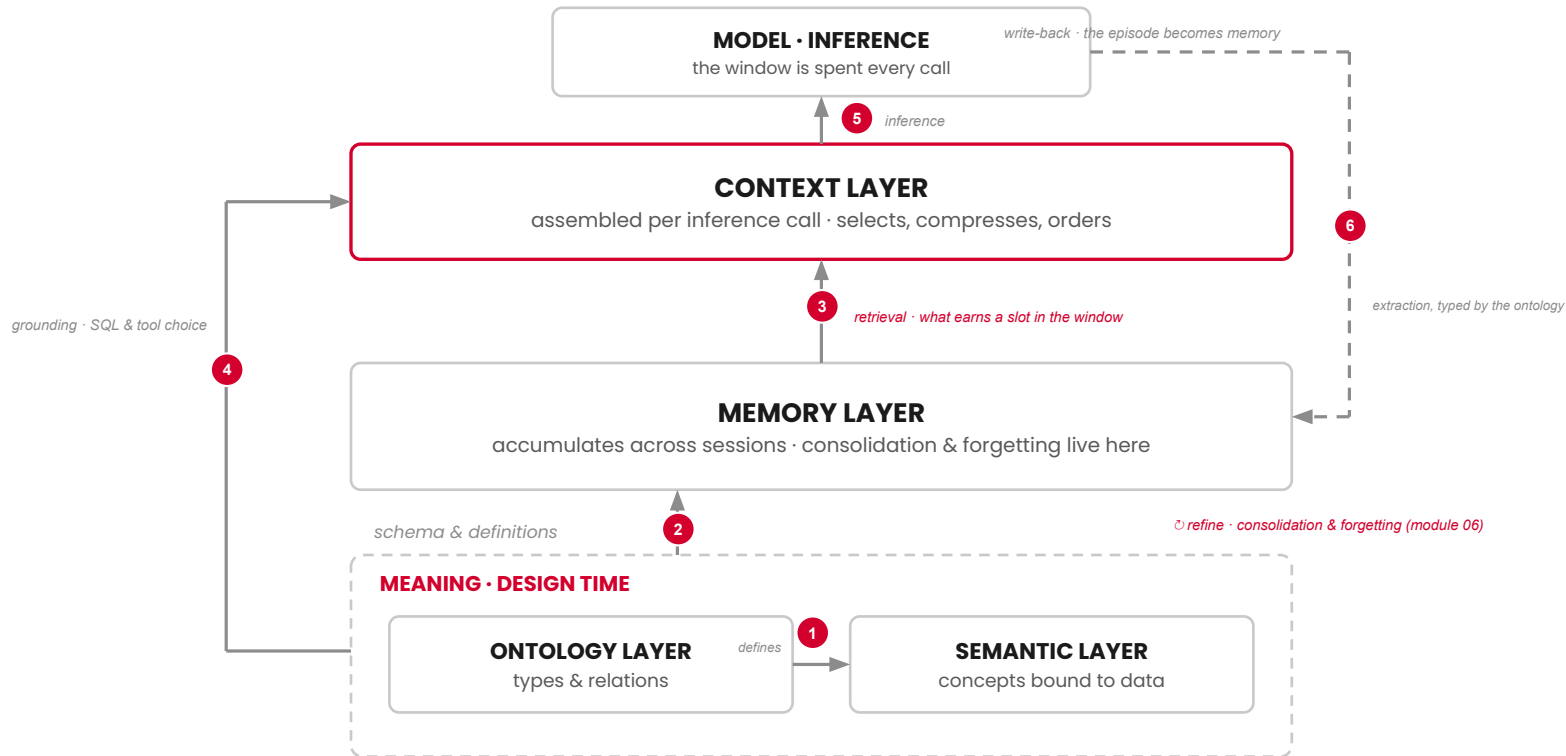
Overview of AI Memory



IN THE LOOP Ontology and semantic are designed, memory accumulates, context is assembled: meaning, then state, then attention.

How The Layers Relate

Overview of AI Memory





What Becomes Advanced?



What Becomes Advanced?



We stop naming components and start proving their operational contracts under failure, ambiguity, scale and change.

01 · MODE-FIRST DESIGN

Select mode before framework

autonomy, horizon, state, verification and delivery become explicit design variables, chosen before any vendor

02 · DURABLE EXECUTION

Checkpoints ≠ side effects

separate the graph's checkpoints from the side-effect ledger; resume safely with idempotency, approval gates and audit evidence

03 · SEMANTIC CONTROL

Meaning is canonical

ground memory, tools, policy and evidence in a semantic and ontology layer, not vector similarity alone

04 · RESEARCH THAT LEARNS

Evidence with provenance

parallel workers curate an evidence pool, consolidate knowledge, and demonstrate measured reuse across sessions

05 · META-HARNESS GOVERNANCE

One contract, many runtimes

a stable memory, policy, approval, verification and event contract around multiple inner harnesses

06 · ONE CORE, MANY SHELLS

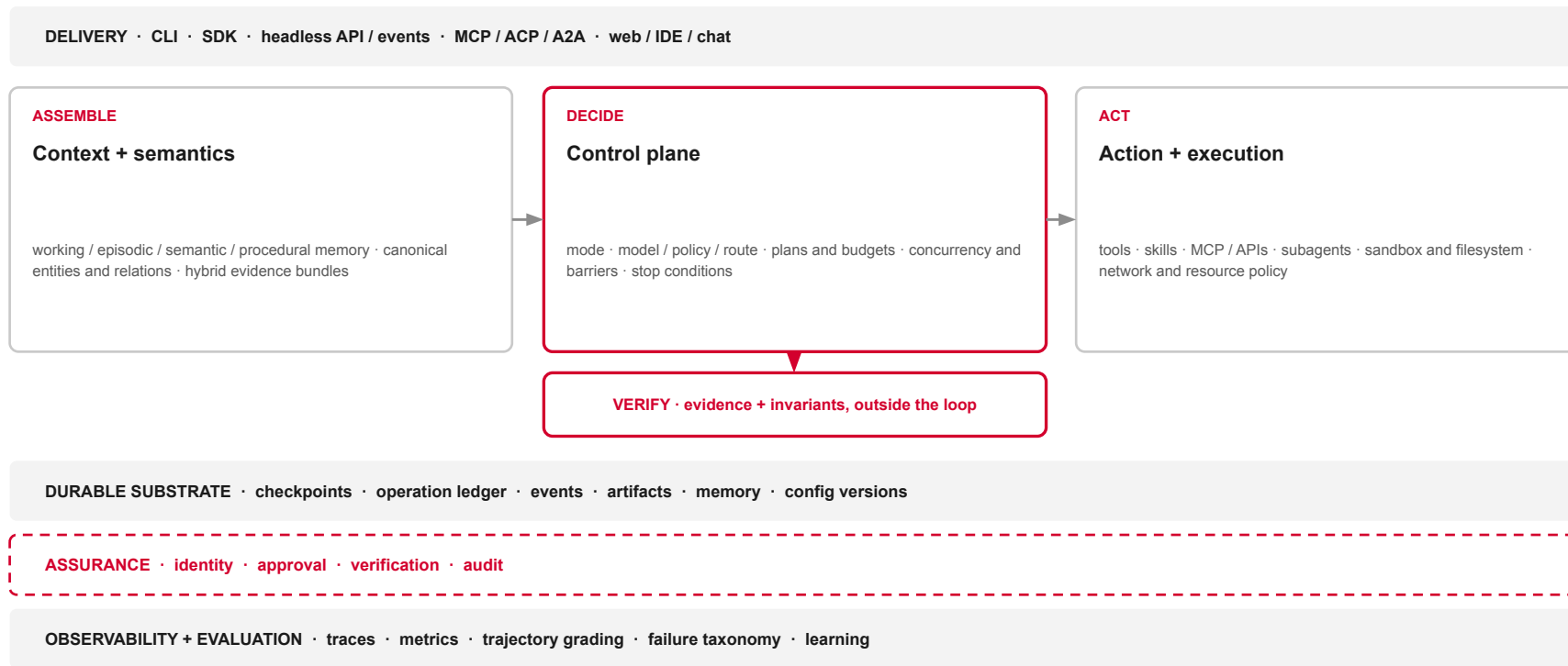
Delivery without divergence

the same governed runtime through CLI, SDK, headless service, MCP, agent protocols and user interfaces

IN THE LOOP Six outcomes, treated as acceptance criteria: by the close, each has evidence behind it, not just a description.



The Advanced Harness Reference Architecture



IN THE LOOP The anchor for everything after: every implementation slide in Parts 03 to 08 maps back to one or more of these planes.

Module 3: Application Mode and Mindset



Three application modes

How an agent is deployed — from **user-in-the-loop** assistance to fully **autonomous** research.

1 Assistant

USER-IN-THE-LOOP

Responds, guides, and remembers across sessions — carrying context forward so each conversation builds on the last.

responds · guides · remembers

2 Workflow **PREDEFINED SEQUENCE**

Embedded in a predefined sequence of steps — orchestrating deterministic logic and tool calls along a structured path.

3 Deep Research **AUTONOMOUS**

Plans, searches, and synthesizes across many sources over multiple reason–act–observe cycles to produce a thorough researched output.



Application Mode Is Not A UI Label

CORE PRINCIPLE

Mode = control-flow ownership + autonomy budget + persistence horizon + evidence obligation.

A chat window can launch a workflow; a headless service can behave like an assistant. Classify the runtime contract, not the surface through which a user sees it.

DESIGN CONSEQUENCE if the statement is true, it must become a host-enforced contract, an observable state transition, or a measurable acceptance criterion: not merely a prompt instruction.

IN THE LOOP Three questions expose the mode: who selects the next action, how long must state survive, who verifies completion.

Three Reference Modes, Eight Design Axes

Mode Contract Matrix · Part 01 · Application Mode



DESIGN AXIS	ASSISTANT	WORKFLOW	DEEP RESEARCH
Next-action owner	Human each turn	Graph / policy	Model + orchestrator
Typical horizon	Seconds to sessions	Minutes to hours	Minutes to days
Control flow	Conversational	Known states and edges	Plan, fan out, adapt
Persistent centre	Identity + cross-session memory	Typed run state + operation ledger	Evidence pool + research memory
Primary risk	Memory leakage · concealed action	Duplicate side effect · stale approval	Bad sources · runaway search
Verification	User judgement + policy	Node and business invariants	Claim-to-evidence coverage
Common delivery	Chat, copilot, voice	API, event, job, UI	UI, async job, report
Stop condition	Turn completed	Terminal state committed	Evidence and synthesis criteria met

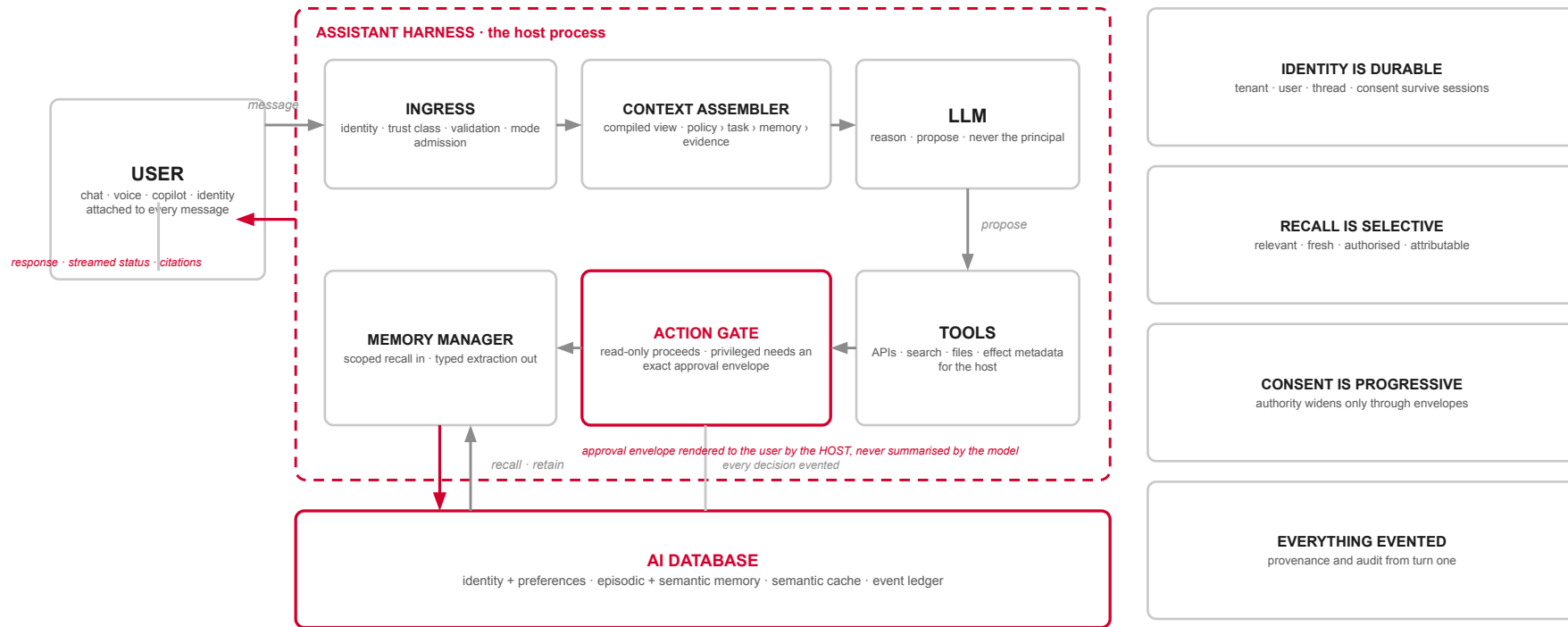
IN THE LOOP Read down a column to design a mode; read across a row to see why copying a chat harness into a workflow fails.



Assistant Mode

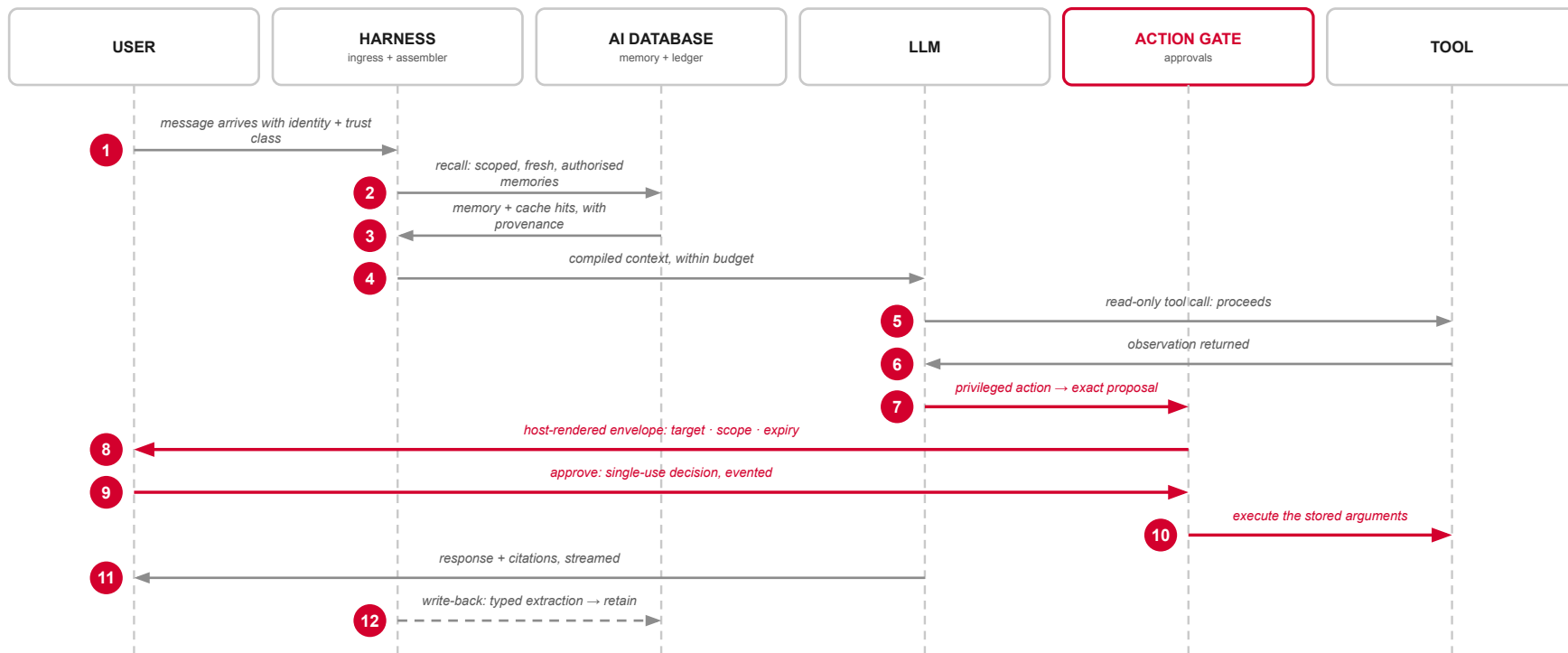


Assistant Mode: Reference Architecture



IN THE LOOP The human steers, the harness remembers, the gate widens authority one envelope at a time: continuity with bounded agency.

Assistant Mode: Execution Sequence



IN THE LOOP Twelve arrows, one turn: recall before reasoning, an envelope before any effect, and a typed write-back before the window resets.



Assistant Mode: Continuity With Bounded Agency

Mode 01 · User In The Loop · Part 01 · Application Mode

The human steers; the harness supplies identity, continuity, consent and safe incremental action.

STATE

Identity is durable

tenant, user, thread, preferences and consent survive sessions without crossing identity boundaries

CONTEXT

Recall is selective

memory enters only when relevant, fresh, authorised and attributable

ACTION

Consent is progressive

read-only help proceeds; writes, purchases and privileged tools require an exact approval envelope

DELIVERY

Streaming matters

the surface projects status, tool activity, interruptions, citations and recovery, not only final text

FAILURE

The hidden hazards

state personalisation, cross-user memory, implicit side effects, and approval dialogs written by untrusted context

PROOF

Acceptance evidence

identity-isolation tests, memory provenance, permission traces, cancellation, and a host-verifiable artifact

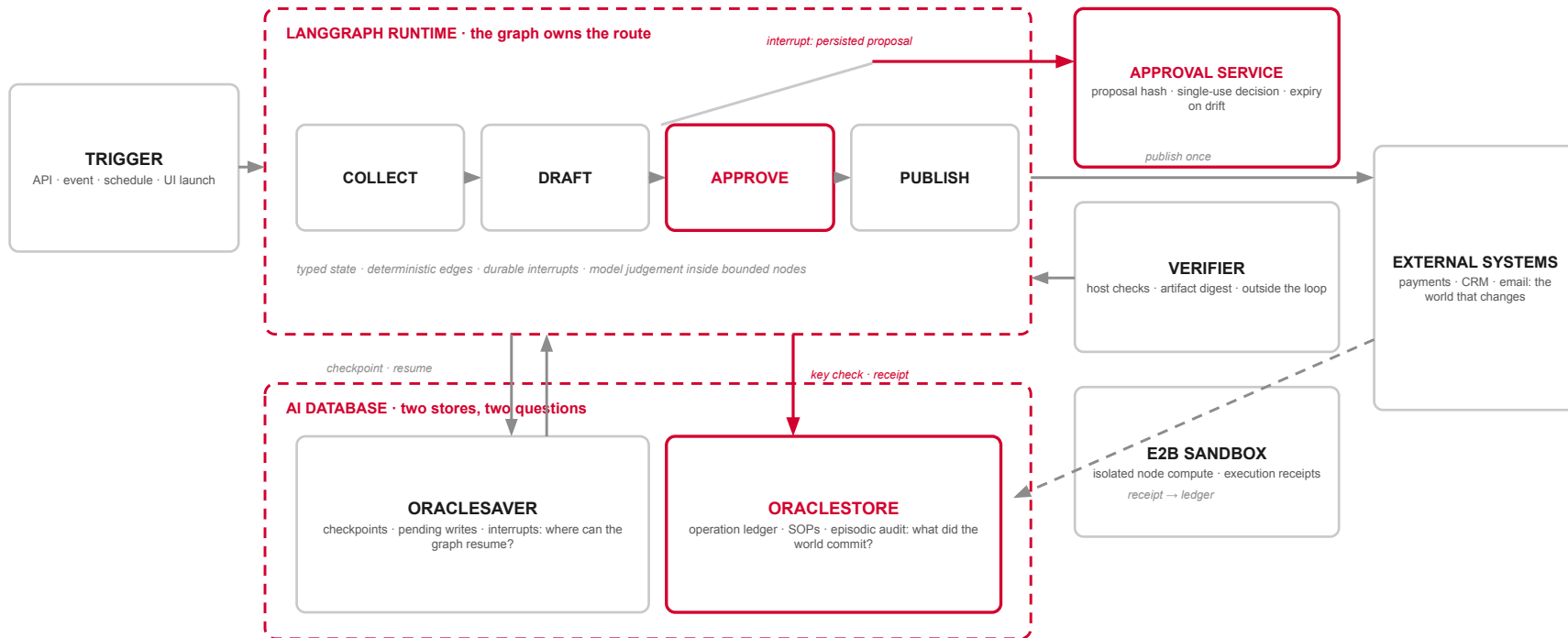
IN THE LOOP Assistant mode can still be high-risk: frequent human interaction does not replace isolation, provenance or exact consent.



Workflow Mode

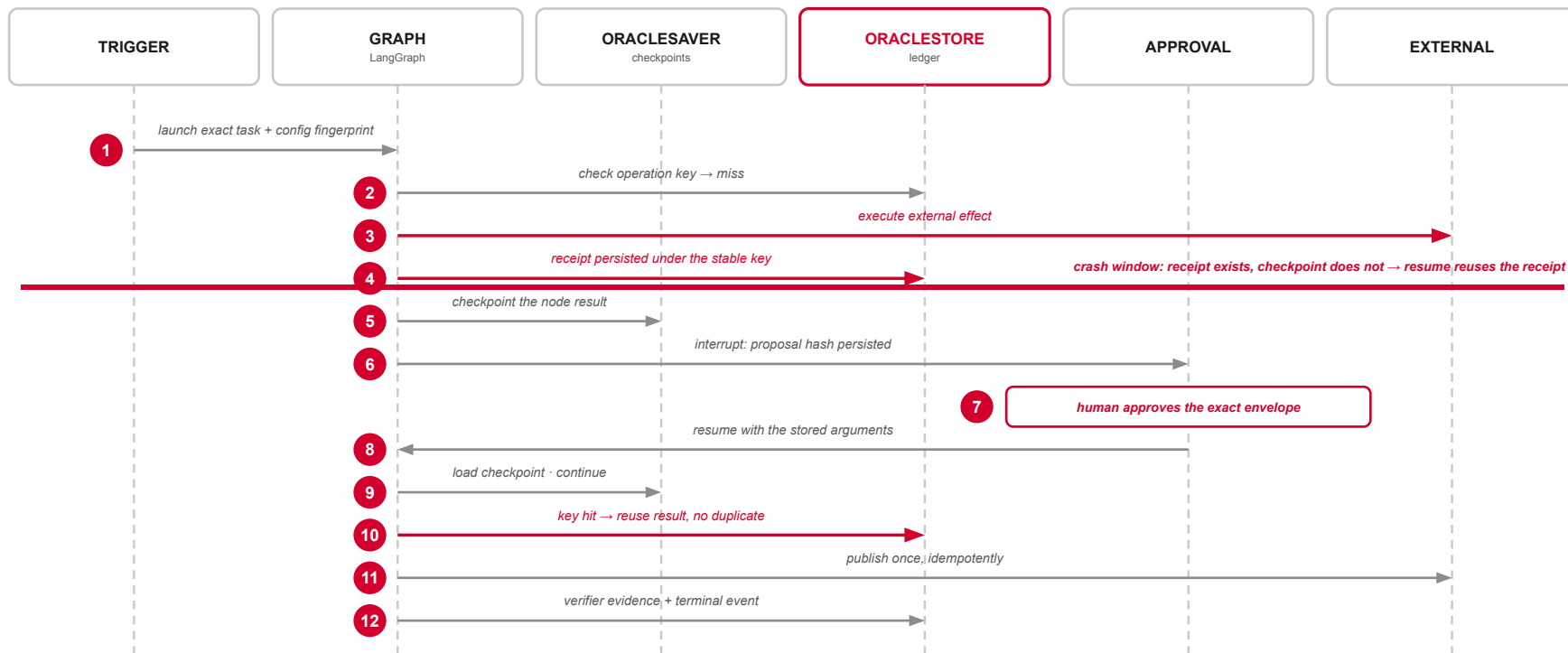


Workflow Mode: Reference Architecture



IN THE LOOP OracleSaver answers where the graph resumes; OracleStore answers what the world committed. Keeping them separate is the whole design.

Workflow Mode: Execution Sequence



IN THE LOOP The graph may run twice; the world changes once: the operation key is what turns a crash into a non-event.



Workflow Mode: Durable State Around Known Work

Mode 02 · Predefined Sequence · Part 01 · Application Mode

The graph owns the route; the model contributes judgement inside bounded nodes.

STATE

Typed checkpoints

persist what resume needs: node, inputs, outputs, errors, versions, and pending external decisions

EFFECTS

Idempotent operations

side effects live in a separate ledger keyed by business identity; a checkpoint is not proof an effect occurred

CONTROL

Deterministic edges

routing, retries, timeouts, compensation and terminal states live in host code, not in prose

HUMAN

Durable interrupts

pause on a persisted proposal; resume the exact stored arguments after approve, edit, reject or timeout

MEMORY

SOP + rule recall

procedural and semantic memory supply current procedures and rules, versioned with provenance

PROOF

Auditable completion

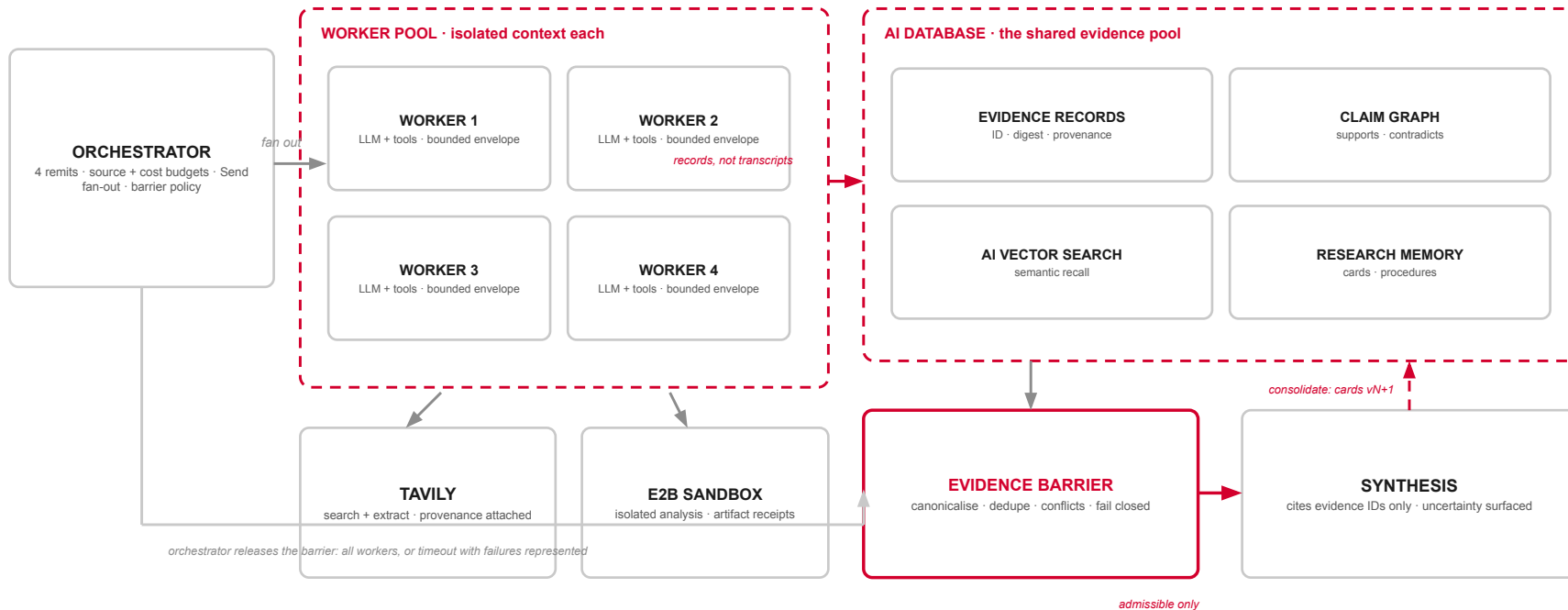
the terminal state binds artifacts, receipts, approvals, verifier results and an immutable timeline

IN THE LOOP Control-flow state and business side effects are different records; most of Part 03 rests on that split.



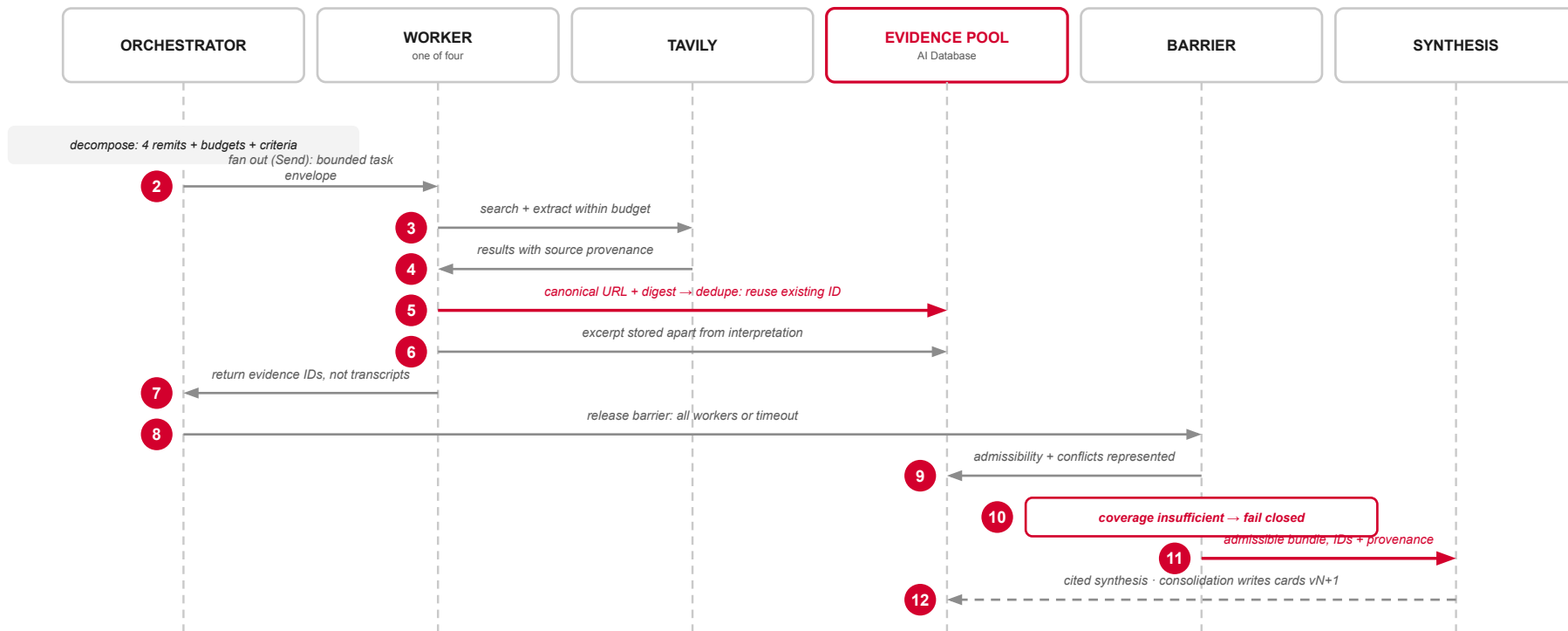
Deep Research Mode

Deep Research Mode: Reference Architecture



IN THE LOOP Workers share evidence through stable records in the pool, never by copying transcripts: provenance survives the fan-out.

Deep Research Mode: Execution Sequence

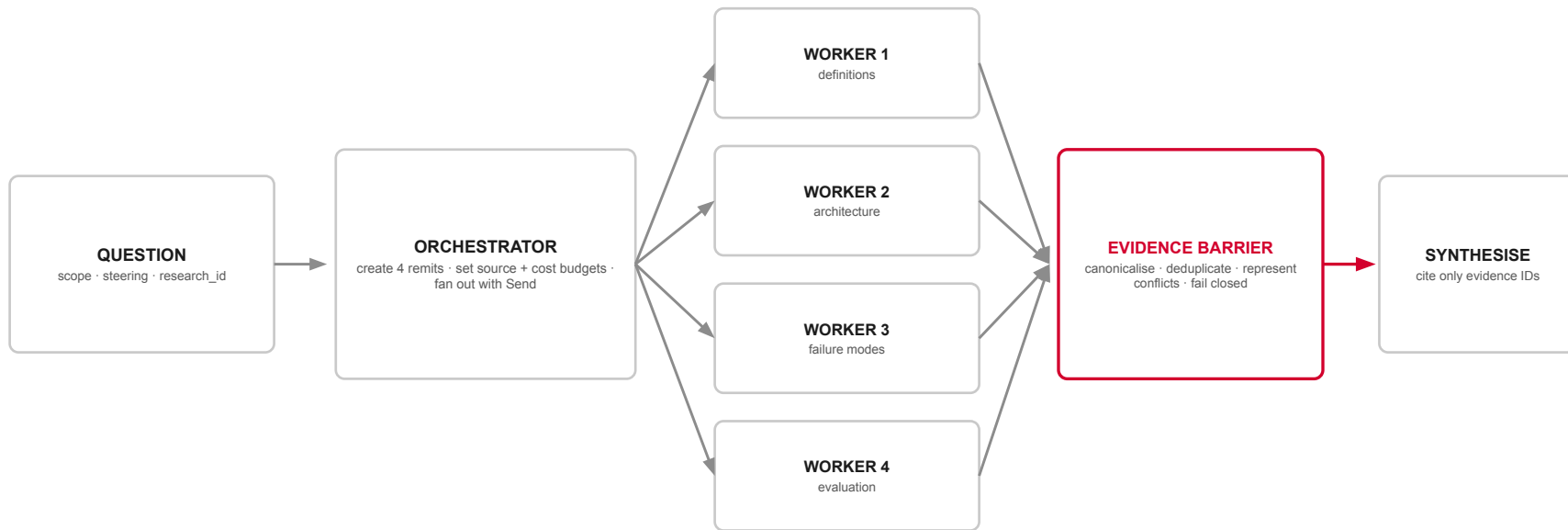


IN THE LOOP Synthesis happens only on the far side of the barrier: everything it cites has an ID, a digest, and a reason to be trusted.



Parallel Deep Research Architecture

Orchestrator · Workers · Barrier · Synthesis · Part 03 · Durable Modes



OracleStore shared evidence pool · Tavily search + extract · Oracle AI Vector Search · session-scoped checkpoints

the barrier does more than wait: it curates, represents failures, and refuses synthesis without admissible evidence



Deep Research Mode: Autonomous Evidence Production



The output is a claim set whose evidence trail, coverage and uncertainty can be inspected.

PLAN

Decompose the question

independent remits, search budgets, source requirements and synthesis criteria, before fan-out

SEARCH

Treat search as a tool

queries, filters, source selection and retries are observable decisions with cost and provenance

EVIDENCE

Curate a shared pool

canonicalise URLs, hash content, deduplicate, timestamp, and separate retrieval from interpretation

BARRIER

Fail closed

no synthesis when workers fail, coverage is insufficient, or required claims lack admissible support

MEMORY

Consolidate after the run

promote verified findings and reusable procedures; decay or quarantine weak, stale or contradicted material

PROOF

Measure learning

track coverage, evidence reuse, prior recall, cache reuse, latency, cost and correctness separately

IN THE LOOP Search quantity is not the metric: admissible evidence and claim coverage are.



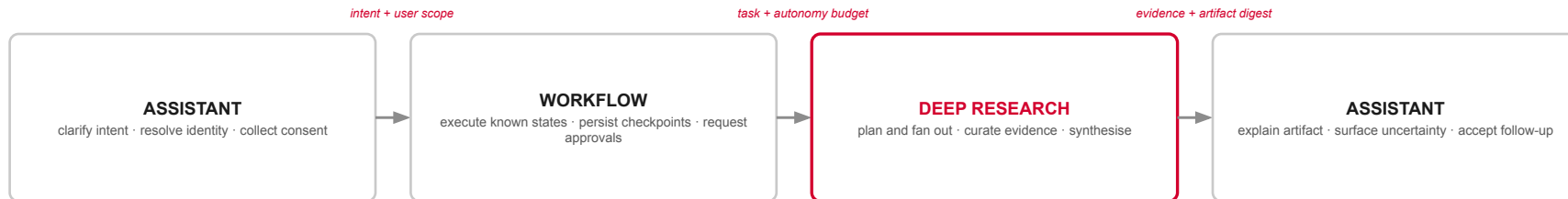
Composability (Modes and Harnesses)

Real Products Compose Modes

Hybrid Application Mode · Part 01 · Application Mode



A single journey can traverse all three modes; the safe design makes each transition explicit and carries a signed envelope.



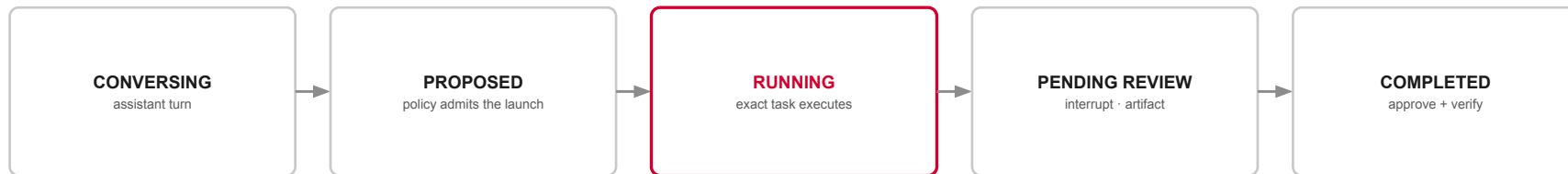
Transition invariant: downstream autonomy may never exceed the authority, data scope, cost, duration and irreversibility budget carried by the upstream envelope.

IN THE LOOP Same journey, three contracts: the envelope is what keeps autonomy from quietly widening between them.



Mode Transitions Are State Transitions

Host-Enforced Transitions · Part 01 · Application Mode



failure paths at every stage: policy or validation failure · reject · cancel · expire

Every arrow writes an event. Every privileged arrow checks a policy. Every terminal state binds verifier evidence.

IN THE LOOP The model may propose a transition; host code validates it and persists the resulting event.



Mindset

Industry Use Cases Are Usually Hybrid



INDUSTRY	HUMAN FRONT DOOR	DURABLE MIDDLE	AUTONOMOUS BRANCH
Financial services	Analyst copilot	KYC / exception workflow	Market and policy research
Healthcare	Clinician assistant	Referral / prior-auth workflow	Evidence review with source gates
Manufacturing	Operator assistant	Incident / maintenance workflow	Root-cause research across logs
Legal	Matter assistant	Review and approval pipeline	Case-law and contract research
Software	IDE / chat copilot	CI remediation workflow	Repository and dependency research
Public sector	Caseworker assistant	Eligibility / escalation workflow	Policy and evidence synthesis

IN THE LOOP Find your row, then find the envelope that crosses from human-facing assistance into durable or autonomous work.



Mode-First Design Worksheet



Answer these before choosing LangGraph, an agent SDK, a coding harness, or a meta-harness.

01 DRIVER · who chooses the next action?

human, deterministic graph, model, policy engine, scheduler, or a combination with explicit transition rules

02 HORIZON · what must survive?

one turn, one run, a crash, an approval delay, a deployment, or months of cross-session memory

03 AUTHORITY · what may the run change?

the exact resources, effects, credentials, network destinations and irreversibility levels

04 EVIDENCE · what proves success?

tests, receipts, source coverage, policy checks, human sign-off, or host recomputation

05 FAILURE · how does recovery work?

retry, resume, compensate, cancel, expire, quarantine or escalate, without duplicating completed effects

06 SURFACE · who consumes it?

CLI user, library caller, webhook, MCP client, another agent, browser UI, chat channel, or scheduled job

Module 3: From Harness to Meta Harnesses





The Harness, Layer by Layer

The six foundational components become nine operational planes once identity, semantics, execution, assurance and observability are made explicit.



From Six Components To Nine Operational Planes

A plane groups components that share a lifecycle and contract. The model sits inside this system; it is not the system boundary.

01

EXPERIENCE + DELIVERY

CLI · SDK · UI · events · channels

02

CONTROL

mode · routing · plans · budgets · stop conditions

03

CONTEXT

instructions · memory recall · compaction · disclosure

04

SEMANTIC + ONTOLOGY

entities · relations · meaning · constraints · provenance

05

STATE + MEMORY

checkpoints · ledgers · artifacts · long-term knowledge

06

ACTION + INTEGRATION

tools · skills · MCP · APIs · agent delegation

07

EXECUTION

sandbox · filesystem · browser · compute · network policy

08

ASSURANCE

identity · authz · approvals · verification · audit

09

OBSERVABILITY + EVAL

events · traces · metrics · trajectory grading · learning



Trust Begins At The Boundary



Every input arrives with identity, channel, modality, trust level and authority scope. Preserve that metadata before flattening to text.

NORMALISE

Canonical event envelope

event_id · principal · tenant · channel · timestamp · modality ·
payload reference · correlation IDs

CLASSIFY

Trust and data class

user instruction, retrieved content, tool output, memory, peer
message and system policy stay distinguishable

VALIDATE

Shape before reasoning

schema checks, size limits, malware scans, MIME and
encoding validation precede context assembly

ROUTE

Mode admission

a request may stay conversational, launch a workflow, start
research, or be rejected before model spend

STREAM

Bidirectional control

progress, partial artifacts, approval requests, cancellation,
steering and terminal status travel both ways

REPLAY

Ingress is reproducible

a privacy-aware envelope lets failures and evaluations replay
the same observed conditions



Context Management Is A Compiled View

The prompt is a transient build artifact assembled from typed sources, not the durable source of truth.

SELECT

Relevance + authority

retrieve only what is relevant to the node, visible to the principal, and valid for the current time

ORDER

Instruction hierarchy

system policy, application policy, task, memory, evidence and tool results keep explicit precedence

DISCLOSE

Progressive loading

expose skill and tool metadata cheaply; load full procedures and schemas only after a match

OFFLOAD

Pointers over payloads

large artifacts move to durable storage; the window carries references, summaries and digests

COMPACT

Keep active invariants

compaction preserves completed actions, IDs, assumptions, blockers, approvals and the next goal

ATTEST

Fingerprint the context

proposals and evaluations record the exact context version, so approval never targets a changed prompt

IN THE LOOP Think compiler: durable sources are parsed, selected, ordered and rendered into a disposable model input.



Memory Is A Governed Lifecycle, Not A Vector Table

Separate what happened, what is believed, how to act, and what is active now; then govern promotion, correction, decay and isolation.

WORKING

Active task state

current goal, plan, constraints, pending calls, scratch facts, and the latest observations

EPISODIC

Events with provenance

what happened, when, to whom, under which policy, with source and confidence preserved

SEMANTIC

Verified domain knowledge

canonical facts resolved against the semantic layer, versioned and open to contradiction

PROCEDURAL

Reusable ways of working

SOPs, skills, recipes, prerequisites, checks, expected effects and evidence of prior success

ASSOCIATIVE

Relationships

links among actors, tasks, artifacts, evidence, policies and memories, not similarity alone

GOVERNANCE

Write policy

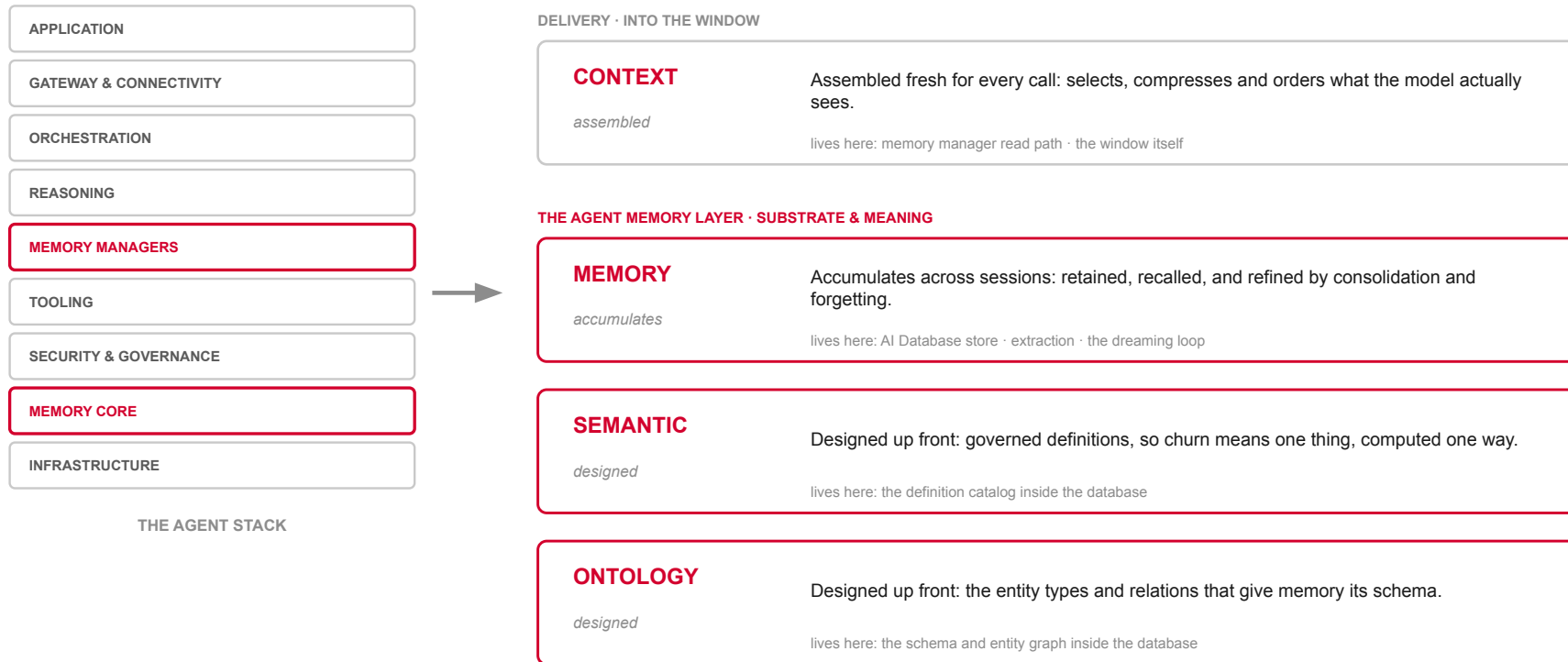
validate, classify, tenant-scope, sign, retain, decay, quarantine, correct and audit every mutation

IN THE LOOP A memory write is a security decision: it can influence every future session that recalls it.



Inside The Agent Memory Layer

Overview of AI Memory



IN THE LOOP Ontology and semantic are designed, memory accumulates, context is assembled: meaning, then state, then attention.



What The Semantic Layer Does At Runtime

Meaning As A Service

An operational service between source systems and the context and action planes, not a catalog UI.

RESOLVE

Canonical identity

map aliases and mentions to stable entity IDs; preserve ambiguity when candidates remain

DEFINE

Business meaning

attach approved definitions, units, ownership, temporal validity, jurisdiction and classification

MAP

Source-to-domain

connect database columns, API fields, documents, metrics and tool parameters to canonical concepts

RETRIEVE

Query expansion

use synonyms, related concepts, entity neighbourhoods and policy filters to improve hybrid retrieval

ROUTE

Capability matching

select tools and skills by semantic effect, prerequisites, resource types and required evidence

EXPLAIN

Traceable grounding

return concept IDs, definition versions, mappings and provenance alongside model-facing text

IN THE LOOP It participates in every assemble, route, retrieve, validate and persist operation.



The Ontology Layer: Formal Domain Meaning

Classes · Relations · Constraints

An ontology states what exists in the domain, how it relates, and which combinations are valid or inferable.

CLASSES

What exists

actor, customer, case, obligation, action, policy, evidence, artifact, approval, tool, transaction

RELATIONS

How things connect

owns · requests · authorises · affects · supports · contradicts · derivedFrom · governedBy · supersedes

CONSTRAINTS

What must be true

cardinality, type, allowed state transitions, required provenance, jurisdiction, evidence obligations

INFERENCE

What follows

a high-impact action on regulated data inherits review and retention duties from its related concepts

VALIDATION

What is admissible

shape languages such as SHACL validate a data graph and return machine-readable violations

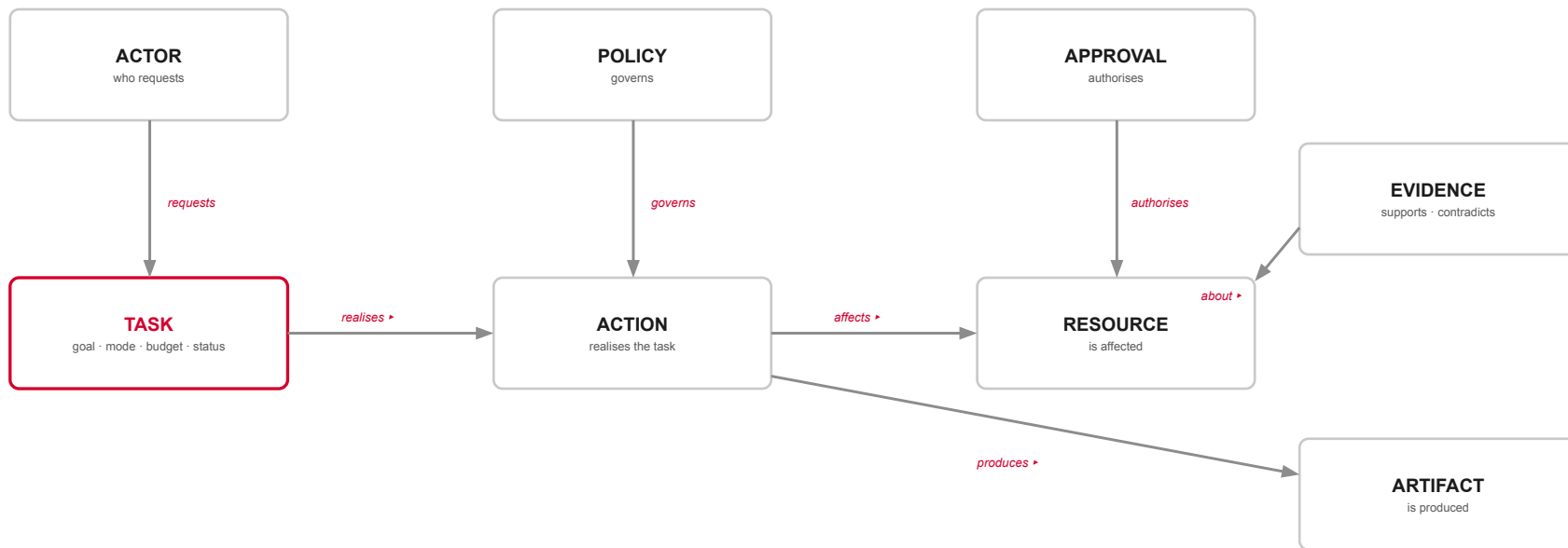
VERSIONING

Meaning changes

concepts, mappings, constraints and policies need effective dates, owners, migrations and compatibility

IN THE LOOP Formal identifiers and relations let independent components agree on meaning and support deterministic validation.

A Canonical Harness Ontology



Every checkpoint, memory, tool call, approval and evaluation result should be relatable to this model through stable identifiers.

IN THE LOOP A pragmatic course ontology: canonical identity and typed relations across state, memory, policy and evidence.



Meta Harness



What A Meta-Harness Owns

Core Principle · Part 04 · Meta-Harnesses

CORE PRINCIPLE

The inner harness owns the loop. The meta-harness owns the operating contract around the loop.

It standardises identity, scoped memory, policy, approvals, budgets, durable events, host verification and evidence across runtimes.

DESIGN CONSEQUENCE if the statement is true, it must become a host-enforced contract, an observable state transition, or a measurable acceptance criterion: not merely a prompt instruction.

IN THE LOOP Not another agent calling agents: a host-side control plane with enforceable contracts and normalised evidence.



The Stable Contract Has Six Jobs

SCOPE

Identity + memory

namespace every task, principal, tenant, project, thread, artifact and memory write; prevent cross-scope recall

SELECT

Capability routing

match task requirements to declared capabilities, health, policy, cost, latency and evaluation history

GOVERN

Policy + approval

compute host-side policy decisions; bind approvals to an exact proposal, context, workspace, budget and verifier

OBSERVE

Normalised events

translate runtime-specific streams into task, model, tool, approval, artifact, usage and verification events

PROVE

Host verification

run acceptance checks outside the model conversation; distinguish proposed completion from verified completion

IMPROVE

Evaluation evidence

compare candidates fairly, record counterfactuals, detect regressions, promote only after safety gates

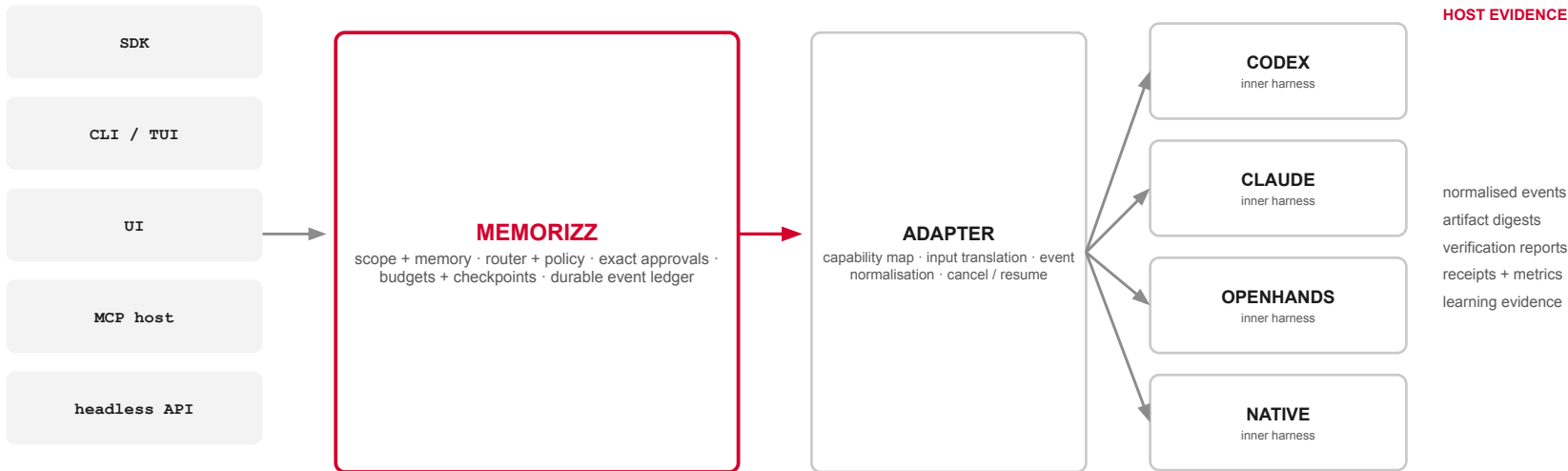
IN THE LOOP For each job, ask: is it described in a prompt, or represented in host state and enforced outside the model?



MemoRizz Meta-Harness Architecture

One Contract · Multiple Runtimes · Part 04 · Meta-Harnesses

CLIENTS



The model may propose. The host authorises, executes privileged boundaries, verifies, records, and decides whether the task is complete.

IN THE LOOP Adapters preserve runtime richness while emitting a common minimum contract: a lowest common denominator is not the goal.



The Governed Operation State Machine

Propose · Authorise · Commit · Verify · Part 04 · Meta-Harnesses



exit paths from any state: DENIED (policy or human rejects) · EXPIRED (context or workspace changed) · FAILED (execution or verifier) · CANCELLED (host revokes)

Invalid transitions are rejected by host code; they are not repaired by asking the model to be careful. Description is not enforcement.

IN THE LOOP Approval expires when the workspace fingerprint changes, and a verifier failure can never be relabelled as success.



Runtime Mode Versus Delegate Mode



Two Integration Patterns · Part 04 · Meta-Harnesses

DESIGN AXIS	RUNTIME MODE	DELEGATE MODE
Loop owner	MemoRizz / native runtime	External harness
Tool mediation	Direct host control	Adapter + runtime boundary
Event fidelity	Full native schema	Depends on runtime stream
Checkpointing	Host-designed	Coordinate host + delegate
Approvals	One authority path	Prevent double prompts and bypass
Best fit	Custom governed workflow	Reuse a mature coding or research harness
Primary risk	More loop implementation	Semantic mismatch · leaky abstraction

IN THE LOOP Delegate mode buys capability and creates two state machines, two cancellation models and two approval systems to reconcile.



Routing Is A Constrained Decision, Not A Model Vibe

Capability + Policy + Evidence · Part 04 · Meta-Harnesses

A route is admissible only when hard requirements pass; optimisation happens inside the admissible set.

HARD GATE

Capability

required tools, modality, context, sandbox, resumability, streaming, structured outputs, protocol support

HARD GATE

Policy

tenant allow-list, data boundary, model and provider restriction, locality, credential scopes, risk class

HARD GATE

Health

runtime available, adapter compatible, model reachable, sandbox warm, dependency versions accepted

OPTIMISE

Quality

task-family evaluation, verifier pass rate, calibration, failure taxonomy and uncertainty, not one leaderboard number

OPTIMISE

Economics

expected cost, latency, queue time, token and tool budget, retry exposure, opportunity cost

RECORD

Route evidence

candidate set, rejected reasons, score version, chosen route, fallback chain, observed outcome

IN THE LOOP Make the router explain why candidates were inadmissible: auditability now, counterfactual data later.



Promotion, Rollback, And Learning



A meta-harness may learn from evidence; it must not silently rewrite production policy from a handful of good traces.

1 · OBSERVE

Collect comparable evidence

versioned task, route, policy, model, environment, outcomes, costs, events, artifacts, verifier reports

2 · PROPOSE

Generate a candidate change

new route weight, memory policy, procedure, verifier, prompt, adapter or capability declaration, with rationale

3 · REPLAY

Offline evaluation

historical and adversarial suites; inspect regressions, safety failures, subgroup effects, confidence intervals

4 · CANARY

Bounded exposure

small traffic slice, strict budgets, enhanced observability, kill switch, predefined abort thresholds

5 · PROMOTE

Human / policy gate

record authoriser, evidence bundle, version, rollout scope, expiry and rollback target

6 · REVERSE

Rollback is a feature

pin prior configuration, revoke the candidate, preserve the incident trace, update the failure taxonomy

IN THE LOOP The evidence may recommend; governance decides. Preserve the losing configuration, and why it lost.



The Harness Landscape



Six Families, Not One Converged Architecture

Landscape Orientation · Part 05 · Harness Landscape

The market converges on components (tools, sessions, sandboxes, events) and diverges on where control and extensibility belong.

KERNEL

Everything is a plugin

DeepSeek Harness exposes models, tools, skills, sessions, storage, sandboxes, loops, schedules and UI through a plugin kernel

GATEWAY

Personal agent runtime

OpenClaw centres a self-hosted gateway joining providers, an agent runtime, tools, channels, sessions and operator surfaces

MINIMAL

Extension-first core

Pi keeps the built-in harness deliberately small and pushes behaviours, tools, UI, persistence and workflows into extensions

META

Harness of harnesses

Omnigent wraps existing coding harnesses behind policy, sandbox, session, collaboration and multi-surface infrastructure

NATIVE

Model-aligned harness

Codex and the Claude Agent SDK expose vendor-aligned agent capabilities, execution patterns and integration surfaces

FRAMEWORK

Composable runtime / SDK

LangGraph, OpenAI Agents SDK, Deep Agents and OpenHands SDK make orchestration primitives directly programmable

IN THE LOOP Four questions for every system: who owns the loop, who owns authority, what can be replaced, how does evidence leave.



DeepSeek Harness: A Plugin Kernel

Ecosystem Case Study · Part 05 · Harness Landscape

DEVELOPER PREVIEW

A small kernel, Cordis, turns core harness capabilities into replaceable plugins.

- official architecture makes models, tools, skills, sessions, sandboxes, storage, loops, scheduling and UI plugin-addressable
- composition is the point: a harness becomes an assembly of capabilities rather than a monolithic agent class
- developer-preview status makes versioning, capability negotiation and operational hardening explicit adoption questions
- the open-source repository is the inspectable contract; marketing labels cannot substitute for it

DELIVERY SURFACES

CLI

web / UI plugins

scheduling

plugin APIs

HARNESS ENGINEERING LESSON plugin systems multiply freedom and compatibility risk: specify lifecycle, isolation, authority, events and semantic versioning for every plugin boundary.

deepseek.com/harness · github.com/deepseek-ai/deepseek-harness

IN THE LOOP Ask what genuinely belongs in the kernel: a plugin boundary is only valuable when its contract covers failure, cancellation, security and observability.



OpenClaw: Gateway + Runtime + Channels

Ecosystem Case Study · Part 05 · Harness Landscape

SELF-HOSTED GATEWAY

OpenClaw separates model and provider, agent runtime, harness, gateway, tools, and user channels.

- the gateway coordinates sessions, routing, tools, devices, channels and operator-facing control surfaces
- runtime documentation treats runtime and harness as distinct layers: useful vocabulary for architecture reviews
- documented surfaces span CLI, web UI, chat channels, mobile and device integrations, and headless execution
- a long-lived personal agent raises durable identity, channel trust, pairing, credential and remote-execution concerns

DELIVERY SURFACES

CLI

web UI

chat channels

headless exec

HARNESS ENGINEERING LESSON multi-channel convenience expands the trust boundary: bind identity, session, device, channel and tool authority instead of treating every message as equivalent input.

docs.openclaw.ai

IN THE LOOP Map one request from a chat channel through the gateway and back: find where authentication ends and authorisation begins.



Pi: Minimal Core, Maximal Extension Surface

Ecosystem Case Study · Part 05 · Harness Landscape

MINIMAL HARNESS

Pi deliberately avoids prescribing every advanced behaviour inside the core harness.

- extensions add tools, commands, UI elements, providers, session behaviour and workflow capabilities
- the project explicitly omits some built-in conventions, such as subagents and plan mode; extensions can supply them
- documented operating surfaces include interactive use, print and JSON output, RPC, and an SDK
- minimalism keeps the trust model legible only if extension permissions, provenance and lifecycle are legible too

DELIVERY SURFACES

interactive CLI

print / JSON

RPC

SDK

HARNESS ENGINEERING LESSON small core does not mean small system: extension governance, capability declarations, isolation and supply-chain evidence become first-class harness components.

pi.dev · github.com/badlogic/pi-mono

IN THE LOOP The advanced lesson is not which philosophy is better: it is where complexity and policy move when the core shrinks.



Omnigent: A Meta-Harness Delivery Platform

Ecosystem Case Study · Part 05 · Harness Landscape

DATABRICKS · OMNIGENT

Omnigent wraps multiple coding harnesses behind a configurable operational and collaboration layer.

- the project describes support for Claude Code, Codex, Cursor, Pi and custom harnesses through configuration
- its scope includes policies, sandboxing, persistent shareable sessions, web and mobile access, and team workflows
- the inner harness can change while the outer session and delivery contract stays comparatively stable
- the official name is Omnigent, not OmniAgent: verify ecosystem claims against primary sources

DELIVERY SURFACES

web

mobile

team sessions

harness adapters

HARNESS ENGINEERING LESSON a meta-harness creates leverage only when adapter semantics are honest: tool events, cancellation, approval, resume, usage and completion cannot be hand-waved into sameness.

github.com/omnigent-ai/omnigent · [Databricks release notes](#)

IN THE LOOP An industry example of the meta-harness shape: compare its envelope with the stricter exact-approval contract from Part 04.

Native Harnesses And Programmable Runtimes

Complementary Architectural Shapes · Part 05 · Harness Landscape



SYSTEM	PRIMARY SHAPE	STRENGTH TO STUDY	INTEGRATION QUESTION
Codex	Coding agent across developer surfaces	Repository work + tool execution	Which host events and policies stay external?
Claude Agent SDK	Programmable Claude Code capabilities	Agent loop + developer controls	How are permissions and sessions mapped?
OpenAI Agents SDK	Tool orchestration SDK	Handoffs, state, tracing	Where do durability and business audit live?
LangGraph	Stateful graph runtime	Explicit state + persistence + interrupts	Which side effects need receipts?
OpenHands SDK	Software-agent SDK / server	Sandboxed execution + agent server	How is artifact verification normalised?
Deep Agents	Composable agent harness	Subagents, tools, context strategies	Which delegations need authority boundaries?

IN THE LOOP Compare control surfaces and contracts, not branding: identify which advanced responsibilities are built in, configurable, or still yours.



Cross-Landscape Design Lessons



What The Systems Reveal · Part 05 · Harness Landscape

RECURRING PATTERN	WHAT IT GIVES YOU	ADVANCED CONSEQUENCE
Plugin / extension systems	Replaceable capability	Lifecycle, trust, compatibility and permission contracts
Persistent sessions	Continuity across turns and devices	Identity binding, retention, recovery, migration
Sandboxes	Bounded execution	Network, secret, filesystem, image and receipt policy
Multiple surfaces	One capability in many contexts	Consistent authz, events, cancellation, accessibility
Subagents / delegation	Parallelism and specialisation	Budgets, scope, provenance, join semantics
Meta-harnessing	Provider and harness portability	Honest adapters and fair evaluation
Open protocols	Ecosystem interoperability	Discovery is not authority; schemas need versioning

IN THE LOOP Locate every consequence in your reference architecture: if no component owns it, the system runs on hope.



Delivery Shapes



A Surface Is Part Of The Trust Boundary



CORE PRINCIPLE

The same capability delivered through a different surface is not automatically the same product.

Identity, consent, streaming, cancellation, accessibility, batching, error semantics and human attention all change with the delivery shape.

DESIGN CONSEQUENCE if the statement is true, it must become a host-enforced contract, an observable state transition, or a measurable acceptance criterion: not merely a prompt instruction.

IN THE LOOP Run one destructive operation through CLI, SDK, chat and a worker: the principal, the approval and the receipt all differ.



Choose The Shape By Operating Context

Delivery-Surface Matrix · Part 06 · Delivery Shapes

SURFACE	HUMAN / CALLER	CONTRACT	PRIMARY RISK
CLI / TUI	Operator or developer	stdin/stdout, exit codes, streams, files	TTY assumptions + unsafe defaults
SDK	Application developer	typed calls, events, exceptions	leaky runtime semantics
Headless API	Remote service	auth, idempotency, async jobs	orphaned work + retries
Event worker	Queue / scheduler	envelope, ack, retry, dead-letter	duplicate side effects
MCP server	Model host	tools, resources, prompts	discovery mistaken for authority
A2A / ACP	Agent or client peer	sessions, messages, updates, tasks	semantic mismatch + delegation
UI / IDE / chat	End user in context	visual state, consent, notifications	hidden scope + attention failure

IN THE LOOP A delivery decision is an architectural decision: write the identity, cancellation, error and receipt contract per surface.



CLI And TUI: The Operator Surface

Composable · Scriptable · Inspectable · Part 06 · Delivery Shapes



A serious CLI has two audiences: a person at a terminal and another program in a pipeline.

INPUT

Explicit invocation

config precedence, working directory, principal, task ID, model and harness selection, budgets, scopes

OUTPUT

Machine mode

versioned JSON / JSONL events, stdout for contract output, stderr for diagnostics, deterministic exit codes

HUMAN

Interactive mode

readable progress, diff and artifact preview, exact approval scope, cancel, pause, resume, accessible colour

LIFECYCLE

Signals and cleanup

SIGINT and SIGTERM handling, child termination, checkpoint before exit, lease release, resumable task ID

SAFETY

No hidden broadening

reject ambiguous paths and globs, redact secrets, make destructive targets concrete, non-interactive CI policy

EVIDENCE

Receipts

route, policy, approvals, tool outcomes, artifact digests, verifier status, usage, terminal reason, stable schemas

IN THE LOOP Demonstrate interactive and machine modes on the same task ID: both views must agree about the terminal state.



SDK, Headless API, And Event Worker

Programmatic Delivery · Part 06 · Delivery Shapes

SDK

Typed local contract

configuration objects, capability discovery, async iteration, cancellation tokens, domain errors, test doubles

HEADLESS

Remote job resource

create, inspect, stream, cancel, resume; idempotency key, optimistic concurrency, authz, retention, webhooks

EVENT

At-least-once reality

unique operation key, inbox / outbox receipts, lease, heartbeat, retry policy, dead-letter, poison handling

BACKPRESSURE

Bound the system

concurrency pools, queue depth, rate limits, per-tenant budgets, admission control, circuit breakers

ERRORS

Stable failure taxonomy

invalid, unauthorised, unavailable, budget exhausted, cancelled, retryable, verifier failed, policy denied

COMPAT

Contract tests

golden events, schema compatibility, replay, cancellation, duplicate delivery, partial streams, older clients

IN THE LOOP The SDK and API expose task lifecycle and evidence, not a prompt endpoint: test duplicates before happy paths.



UI, IDE, And Chat Are Contextual Shells

Human-In-The-Loop Delivery · Part 06 · Delivery Shapes

UI

Visible state

phase, plan, active tools, scopes, budgets, evidence, artifacts, uncertainty, blocked reason, terminal verification

APPROVAL

Decision quality

exact target and consequence, relevant diff, credential and network scope, expiry, verifier, alternatives

IDE

Workspace binding

repository root, branch, dirty state, file provenance, diagnostics, test results, diff ownership, trust state

CHAT

Channel identity

user, tenant, room, bot identity, device, mention semantics, attachments, retention, channel authority

ATTENTION

Async notifications

notify on decisions and terminal states, not every token; deduplicate, route by urgency, keep the trail

ACCESS

Accessibility

keyboard operation, screen-reader labels, non-colour status, readable diffs, localisation, safe mobile approvals

IN THE LOOP A polished chat answer can conceal an unverified task: visually separate narration, tool evidence, policy and verifier outcome.



Protocol Map: MCP, A2A, And ACP

Interoperability Is Layered · Part 06 · Delivery Shapes

MCP · MODEL ↔ CONTEXT / CAPABILITY SERVER

prompts · resources · tools

capability exposure and context exchange; the host retains control over sampling, roots, permissions and use

A2A · AGENT ↔ AGENT

identity · task · messages · artifacts

agent interoperability and delegation; application authority and evidence semantics still require policy

ACP · EDITOR / CLIENT ↔ CODING AGENT

session · updates · permissions · cancel

a client-agent lifecycle protocol; adapter design must map runtime events without erasing meaning

IN THE LOOP Protocol support answers whether these parties can communicate; it never answers whether this operation should be allowed.

One Core, Many Shells

Hexagonal Delivery Architecture · Part 06 · Delivery Shapes



Shells translate identity, input, events, approvals and output. They do not reimplement state transitions or invent their own truth about completion.

IN THE LOOP Ports and adapters for agents: six shells, one lifecycle, one truth about completion, owned by the core.

Module 4: Evaluation and Security





Name The Experimental Unit



CORE PRINCIPLE

A harness benchmark score belongs to a configured system, not to the model alone and not to the harness alone.

Result = task release × environment × model snapshot × harness and adapter × tools × budgets × policy × sampling × verifier × repetitions.

DESIGN CONSEQUENCE if the statement is true, it must become a host-enforced contract, an observable state transition, or a measurable acceptance criterion: not merely a prompt instruction.

IN THE LOOP Read a leaderboard row as an experimental configuration: any absent variable makes the result unsafe to attribute.



Observability And Evaluation: Inspect The Trajectory



Final-answer scoring hides duplicate work, unsafe attempts, bad routing and lucky guesses. Instrument the run.

TRACE

End-to-end lineage

one trace binds task, mode, model, config, plan, tools, subagents, approvals, artifacts and terminal state

EVENT

Normalised lifecycle

planned · started · observation · tool proposed · approved · result · verified · cancelled · failed · completed

METRIC

Operational signals

latency, cost, tokens, calls, retries, cache hits, fan-out, evidence reuse, verifier failures, approval time

OUTCOME

Task-native success

business or benchmark criteria stay primary; style and judge preference are separate constructs

TRAJECTORY

Behavioural assertions

required and forbidden tools, action order, grounding, idempotency, policy adherence, stop behaviour

LEARNING

Failure taxonomy

classify model, context, tool, data, policy, orchestration, sandbox and evaluator failures before changing anything



How Terminal-Bench Works

Task → Sandbox → Agent → Verifier · Part 07 · Evaluating Harnesses



Terminal-Bench 2.0 introduced 89 hard, workflow-inspired terminal tasks with unique environments, human-written solutions and comprehensive verification. The benchmark supplies task and judge; Harbor supplies the repeatable runtime; your adapter supplies the harness under test.

IN THE LOOP The oracle solution validates task solvability; it is never context given to the tested agent.



Inside One Harbor Trial

Reproducible Run Lifecycle · Part 07 · Evaluating Harnesses



01 RESOLVE

pin dataset release, task, environment provider, agent adapter, model and trial arguments

02 PROVISION

create the environment, validate prerequisites, install the adapter, start the limits

03 EXECUTE

give the instruction; capture commands, interactions, timing, usage and the full trajectory

04 VERIFY

run task-owned tests against the resulting environment; write reward outside the agent

05 PERSIST

store trial config, result, agent log, verifier log, reward and trajectory under one job identity

06 AGGREGATE

compute task and suite results across attempts; analyse distributions, cost, latency, exclusions

IN THE LOOP harbor run -d dataset@version -a agent -m model -k attempts: smoke-test a tiny subset before the suite, and keep keys out of everything.



What Terminal-Bench Tells You, And What It Does Not



Interpretation Boundary · Part 07 · Evaluating Harnesses

AXIS	SUPPORTS EVIDENCE ABOUT	DOES NOT ESTABLISH
Task outcome	Verifier-scored terminal end state	Correctness outside verifier coverage
Long-horizon work	Realistic multi-step navigation	Reliability in every industry workflow
Model + harness	The exact configured pair	A model-only or harness-only causal claim
Tool use	Effective command-line interaction	Safety of privileged production tools
Efficiency	Cost, tokens and time when recorded	A universal economic optimum
Reproducibility	Pinned releases, logs, attempts	Freedom from contamination or drift
Security	Failures the included tasks expose	A complete agent threat assessment

IN THE LOOP A benchmark is an instrument with a construct and a coverage boundary: report version, exclusions, attempts and uncertainty.

Evaluate Every Harness Plane

Component-Level Test Map · Part 07 · Evaluating Harnesses



PLANE	EVIDENCE	REPRESENTATIVE TESTS
Ingress	Parsed request + provenance	malformed input · injection · identity mismatch
Context	Selected items + token budget	needle / decoy · exclusion · compaction fidelity
Memory	Scoped reads and writes + lineage	cross-tenant leak · poison · stale, delete, consent
Semantics	Entity and claim IDs + ontology version	alias collision · constraint violation · migration
Control	State transitions + checkpoints	crash and resume · cancel race · retry duplicate
Action	Tool call + receipt	schema fuzz · least privilege · timeout · side effect
Sandbox	Image, scope, network, resource evidence	escape · egress · secret spill · exhaustion
Verification	External report + artifact digest	false pass or fail · adversarial artifact · nondeterminism
Delivery / ops	Events, SLOs, approvals, UX	schema compatibility · load · alerting · human factors

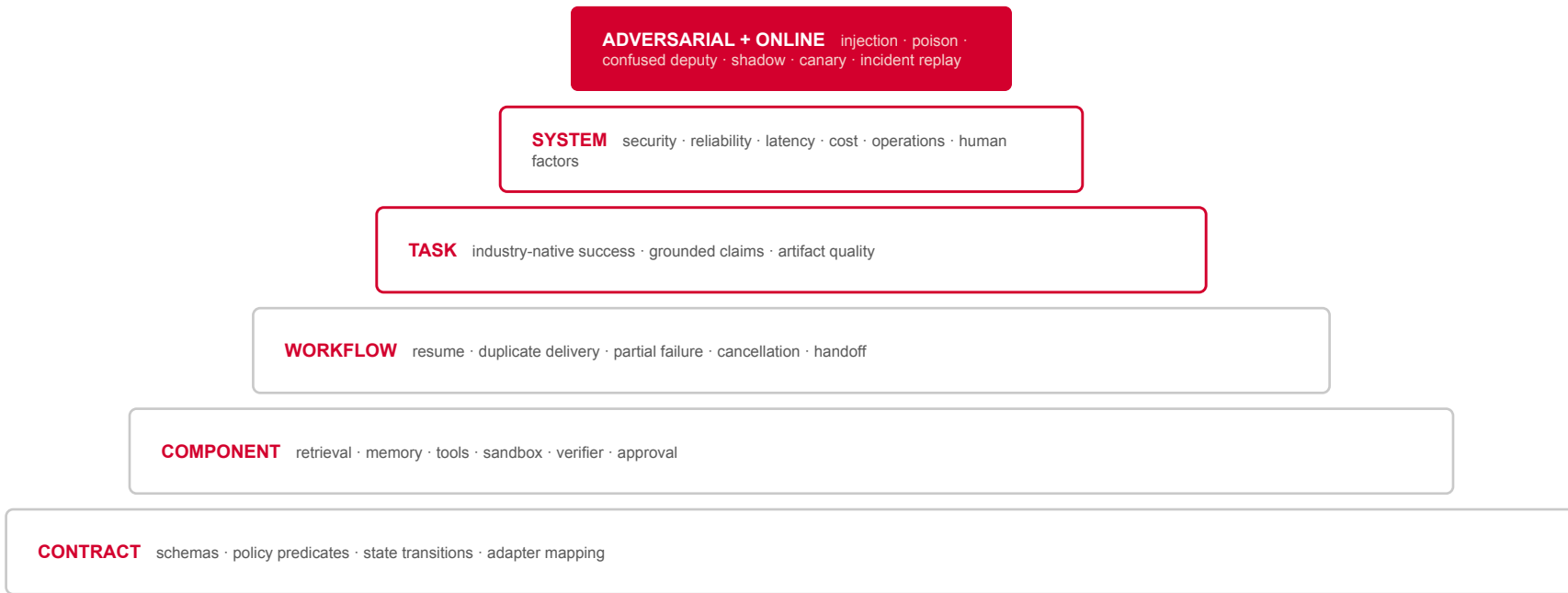
IN THE LOOP Terminal-Bench supplies task-level evidence; the other planes need deterministic tests, fault injection and live monitors.



The Harness Evaluation Pyramid

Prove From Component To Outcome · Part 08 · Production Proof

build upward: deterministic contracts first, live evidence last



IN THE LOOP A model benchmark does not evaluate the harness: build from deterministic contracts upward, then add adversarial and online evidence.



Security



Threat-Model The Whole Harness

INGRESS

Instruction injection

user text, retrieved pages, tool output, attachments, MCP servers, memory or agent messages redefining authority

CONTEXT

Sensitive-data spill

over-broad retrieval, cross-tenant memory, secrets in traces, provider data-policy mismatch, stale access

ACTION

Excessive agency

confused-deputy tools, broad credentials, shell injection, path escape, uncontrolled network, unsafe delegation

STATE

Memory poisoning

untrusted content promoted to durable truth, alias collisions, fabricated provenance, malicious procedures

CONTROL

Lifecycle abuse

approval replay, resume under changed context, duplicate side effects, budget bypass, cancellation races

SUPPLY

Extension compromise

malicious skills, plugins or MCP servers, dependency confusion, mutable images, unsigned artifacts

IN THE LOOP For each threat: a preventive control, a detective signal, a containment boundary, and a recovery test. The model refusing is one weak layer.



Sandbox Policy Is More Than Runs In A Container



IMAGE

pinned template, package allow-list, SBOM, non-root user, immutable base

FILES

ephemeral workspace, explicit mounts, path policy, quotas, artifact export by allow-list

NETWORK

deny by default, domain and port egress, DNS policy, proxy receipts

SECRETS

scoped injection, short TTL, process-bound, redaction, revocation

COMPUTE

CPU, memory, process, disk and wall-time limits, concurrency lease

EVIDENCE

command + cwd + image, exit code, stdout and stderr digests, files changed, usage

IN THE LOOP The sandbox constrains execution; host policy constrains whether execution may begin; verification constrains whether its output counts.

Production Scorecard And Rollout Gates



AXIS	EVIDENCE	EXAMPLE GATE	ABORT SIGNAL
Task quality	Task-native verifier + review	No material regression	Critical task failure
Grounding	Claim-to-evidence coverage	Required claims sourced	Fabricated or missing source
Safety	Policy and adversarial suite	Zero critical violation	Scope, secret or authz breach
Durability	Fault injection + replay	No duplicate side effect	Irrecoverable or double commit
Operations	SLOs, saturation, error taxonomy	Within error budget	Queue or failure-rate threshold
Economics	Cost and latency distribution	Budget by task class	Runaway spend or tail latency
Human factors	Approval quality + UX study	Comprehensible decisions	Approval fatigue or dark pattern

IN THE LOOP Pre-register the gates and abort thresholds: a canary is governance only when something can actually stop it.